Software Requirements Specification

for

Eco-Volunteer

Version <1.0>

Prepared by

Group Name: GROUP 16

Student Name	Student ID	Student Email ID
D. Lalitha Manasvini	SE23UCSE051	Se23ucse051@mahindrauniversity.edu.in
P.Srija	SE23UARI094	Se23uari094@mahindrauniversity.edu.in
S.Sneha	SE23UARI121	Se23uari121@mahindrauniversity.edu.in
P.Harika	SE23UARIO97	Se23uari097@mahindrauniversity.edu.in
P.Ananya	SE23UARI087	Se23uari087@mahindrauniversity.edu.in
Akshaya	SE23UARI007	Se23uari007@mahindrauniversity.edu.in
Venkata	SE23UARI035	Se23uari035@mahindrauniversity.edu.in
Akarshan	SE23UARI151	Se23uari151@mahindrauniversity.edu.in

Instructor:	*Nartakannai*
Course:	Programing workshop/Software Engineering
Lab Section:	*First Floor, Wing 2, IT 2 Building*
Teaching Assistant:	*NA*
Date:	9 March 2026

Contents

CONTENTS ..... **II**

REVISIONS ..... **III**

1 INTRODUCTION **1**

1.1 DOCUMENT PURPOSE 1
1.2 PRODUCT SCOPE 1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW 2
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS 2
1.5 DOCUMENT CONVENTIONS 3
1.6 REFERENCES AND ACKNOWLEDGMENTS 4

2 OVERALL DESCRIPTION **4**

2.1 PRODUCT OVERVIEW 4
2.2 PRODUCT FUNCTIONALITY 5
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS 6
2.4 ASSUMPTIONS AND DEPENDENCIES 6

3 SPECIFIC REQUIREMENTS **7**

3.1 EXTERNAL INTERFACE REQUIREMENTS 7
3.2 FUNCTIONAL REQUIREMENTS 8
3.3 USE CASE MODEL 10

4 OTHER NON-FUNCTIONAL REQUIREMENTS **13**

4.1 PERFORMANCE REQUIREMENTS 13
4.2 SAFETY AND SECURITY REQUIREMENTS 14
4.3 SOFTWARE QUALITY ATTRIBUTES 15

5 OTHER REQUIREMENTS **16**

APPENDIX A – DATA DICTIONARY **17**

APPENDIX B - GROUP LOG **18**

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Draft 1	Manasvini, Srija	Initial draft of the SRS document created	09/03/26
Draft 2	Harika, Sneha	Added use case diagrams and their description	10/03/26
Draft 3	Ananya, Akshaya	Revised and finalized entire SRS document	11/03/26

1 Introduction

The Eco-Volunteer platform is a digital system designed to connect volunteers with environmental activities such as tree plantation drives, clean-up campaigns, and sustainability initiatives. The platform enables users to easily discover nearby eco-activities, register for events, track their participation, and contribute to environmental causes. It also supports collaboration between volunteers and environmental organizations by providing a centralized space for managing and promoting activities. Through features like event discovery and interactive maps, users can quickly locate and join initiatives in their area. The system aims to encourage community participation and increase awareness of environmental sustainability efforts. This section introduces the purpose, scope, intended audience, and key terminology used throughout this Software Requirements Specification (SRS) document.

1.1 Document Purpose

This document describes the functional and non-functional requirements of the Eco-Volunteer system. It outlines the expected behavior of the platform, including system interfaces, constraints, and performance requirements. The document serves as a structured guide for understanding how the system should operate and what features it must support. It helps ensure that all aspects of the platform are clearly defined before development and implementation. The goal is to provide developers, project managers, instructors, and stakeholders with a clear understanding of the system requirements so the system can be designed and implemented effectively.

1.2 Product Scope

Eco-Volunteer is a **web-based and mobile-friendly platform** that enables volunteers to participate in environmental initiatives. The system provides location-based discovery of eco-events, volunteer registration, event management, and progress tracking.

The system aims to:

- Encourage environmental participation
- Connect volunteers with NGOs and eco-organizations
- Provide real-time event information and location of programs, events, drives
- Track volunteer contributions and award badges

Benefits include increased environmental awareness, easier coordination of eco-activities and improved community participation.

1.3 Intended Audience and Document Overview

This document is intended for:

- **Developers** – to implement system features
- **Project Managers** – to monitor project progress
- **Instructor / Evaluators** – to evaluate project design
- **Testers** – to validate system functionality
- **Users / Volunteers** – to understand system capabilities and connect with events
- **Organisations** – to provide information of programs/events being conducted

Document Structure:

Section	Description
Section 1	Introduction and background
Section 2	Overall system description
Section 3	Detailed functional requirements
Section 4	Non-functional requirements
Section 5	Additional system requirements

1.4 Definitions, Acronyms and Abbreviations

Acronym / Term	Definition
API	Application Programming Interface; a set of rules that allows different software systems to communicate with each other.
CRUD	Create, Read, Update, Delete; the four basic operations performed on data in a database.
DB	Database; an organized collection of structured information stored electronically and accessed by the application.
Eco-Event	An environmental activity such as tree plantation, beach cleanup, recycling drive, or awareness campaign organized through the Eco-Volunteer platform.
Eco-Volunteer Platform	The proposed web-based system that connects volunteers with environmental events and organizations.
GPS	Global Positioning System; a satellite-based navigation system used to determine a user's geographic location for location-based event discovery.
NGO	Non-Governmental Organization; an organization that may create and manage environmental events on the platform.
REST	Representational State Transfer; an architectural style used for designing networked applications and APIs.
SRS	Software Requirements Specification; a document that describes the functional and non-functional requirements of the software system.
UI	User Interface; the visual elements (screens, buttons, menus) through which users interact with the system.

UML	Unified Modeling Language; a standardized modeling language used to visualize system architecture and design.
User	Any individual interacting with the system, such as volunteers, organizers, or administrators.

1.5 Document Conventions

This document follows the **IEEE Software Requirements Specification (SRS) standard** to ensure clarity, consistency, and ease of understanding for all stakeholders involved in the Eco-Volunteer system development.

1.5.1.1 Formatting Conventions

- The document uses **Arial font, size 11 or 12** for all text content.
- Section headings and subsection headings follow the **hierarchical numbering format** specified in the IEEE SRS template (e.g., 1, 1.1, 1.2, etc.).
- All text in the document is **single-spaced** with **1-inch margins** on all sides.
- Tables are used where appropriate to organize definitions, requirements, and data elements clearly.
- Comments or notes within the document are written in **italics** to differentiate them from the main content.

1.5.1.2 Naming Conventions

- Functional requirements are identified using the format **F1, F2, F3, …**.
- Use cases are labeled using the format **U1, U2, U3, …**.
- System components, modules, and actors are named using **clear and descriptive identifiers** to avoid ambiguity.

1.5.1.3 Requirement Conventions

- The keyword **"shall"** is used to indicate mandatory system requirements.
- The keyword **"should"** is used to indicate recommended but non-mandatory behavior.
- The keyword **"may"** is used to describe optional functionality.

These conventions help maintain consistency and ensure that all stakeholders interpret the system requirements accurately throughout the development lifecycle.

1.6 References and Acknowledgments

References used for the development of this system:
IEEE SRS Documentation Standard
React (Leaflet)
Software Engineering course materials

2 Overall Description

2.1 Product Overview

The **Eco-Volunteer Platform** is a new, self-contained web-based system designed to connect volunteers with environmental activities organized by NGOs, community groups, and environmental organizations. The platform serves as a centralized system where volunteers can discover eco-events, register for participation, track their contributions, and collaborate with organizations that promote sustainability and environmental protection.

Currently, many environmental initiatives rely on scattered communication methods such as social media posts, manual registrations, or messaging groups, which makes it difficult for volunteers to find nearby opportunities and for organizers to manage participants effectively. The Eco-Volunteer system addresses this problem by providing a structured digital platform that integrates event management, volunteer coordination, location-based discovery, and communication tools in a single system.

The system will interact with several external components including map services for location-based event discovery, a database for storing user and event data, and notification services to inform volunteers about upcoming activities. Volunteers will access the system through a web or mobile interface, while organizers will use an administrative interface to create and manage eco-events. The platform will also support integration with external APIs for maps and location services to help users easily identify environmental activities near them.

Overall, the Eco-Volunteer platform will act as an intermediary between volunteers, organizers, and environmental initiatives, improving participation, simplifying event coordination, and promoting community engagement in sustainability efforts.

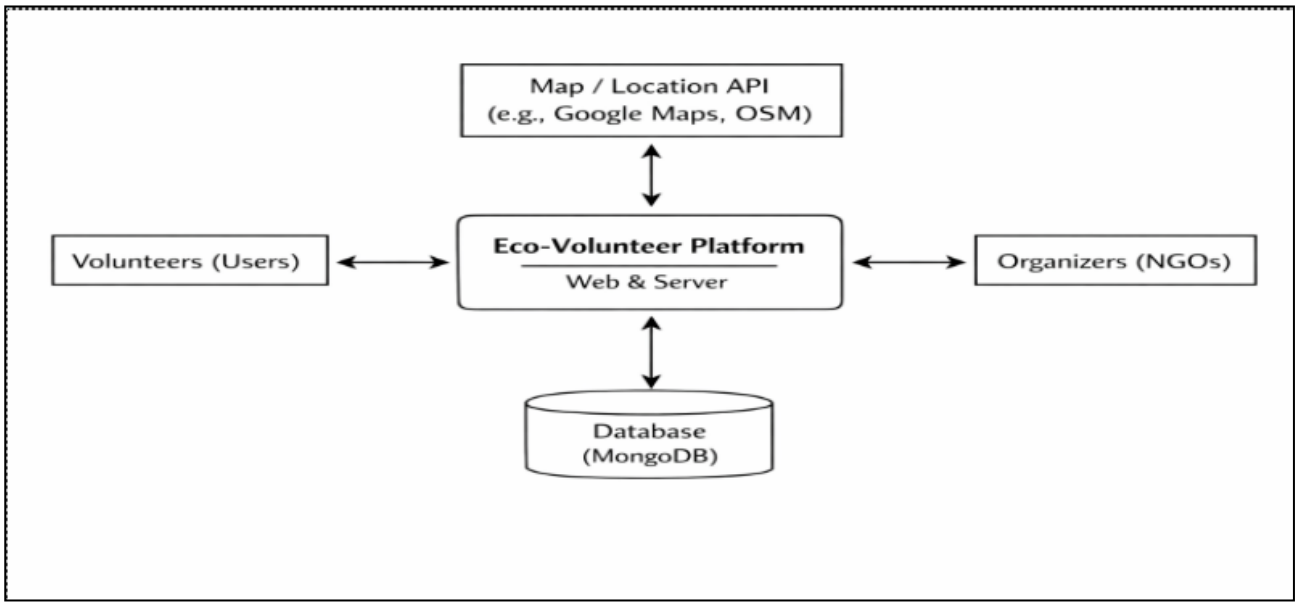

Fig 1: Overview of our product interactions with the environment

Explanation of Components

1) **Volunteers (Users)** – Individuals who register on the platform to find and participate in eco-events.
2) **Organizers (NGOs)** – Environmental organizations or community groups that create and manage eco-events.
3) **Eco-Volunteer Platform** – The main system responsible for managing users, events, registrations, and communication.
4) **Map / Location** – External services used to display event locations and help users discover nearby activities.
5) **Database** – Stores user information, event details, and participation records.

2.2 Product Functionality

The Eco-Volunteer platform provides a centralized system that enables volunteers and environmental organizations to collaborate in environmental activities. The system allows users to discover eco-events, register for activities, manage events, and track participation. The following list summarizes the major functions of the Eco-Volunteer system at a high level.

- **User Registration and Authentication**
  The system allows volunteers and organizers to create accounts and securely log in to access the platform.
- **User Profile Management**
  Users can create and update their personal profiles, including contact details, interests, and volunteering history.

- **Eco-Event Discovery**
  Volunteers can browse and search for environmental events such as clean-up drives, tree plantations, and awareness campaigns.
- **Location-Based Event Search**
  The system displays nearby eco-events using map services and location data.
- **Event Registration**
  Volunteers can register to participate in eco-events organized by NGOs or community groups.
- **Event Creation and Management**
  Organizers can create, update, and manage eco-events, including event descriptions, locations, schedules, and participant limits.
- **Volunteer Participation Tracking**
  The system records volunteer participation in events and maintains a history of completed activities.
- **Notification and Alerts**
  The platform sends notifications to users regarding upcoming events, new events in their area, or updates related to registered activities.
- **Event Reporting and Statistics**
  Organizers and administrators can view reports on volunteer participation, event attendance, and environmental impact.
- **Administration and System Monitoring**
  System administrators can manage users, events, and system data to ensure proper functioning of the platform.

2.3 Design and Implementation Constraints

The following constraints affect system development:

- The system will be developed using **React for frontend**
- Backend will use **Node.js with Express**
- Database will use **MongoDB**
- Location services will use **React (Leaflet)**
- The system must follow **UML modeling standards**
- Software design will follow the **COMET method**

2.4 Assumptions and Dependencies

Assumptions

- Users have stable internet access to use the web-based platform.
- GPS or location services are enabled on user devices for location-based event discovery.
- Environmental organizations and NGOs are willing to register and create eco-events on the platform.
- Users have basic digital literacy to navigate the web interface and register for events.
- The platform will be accessed primarily through modern web browsers such as Chrome, Edge, or Firefox.

- Volunteers will provide accurate personal information during registration.
- Event organizers will update event details and schedules regularly to ensure accurate information for volunteers.
- Users will comply with platform guidelines and use the system for legitimate environmental activities.

Dependencies

- MongoDB database service for storing user profiles, event details, and participation records.
- Authentication services for user login and identity verification.
- Cloud hosting or server infrastructure to run the web application and manage system requests.
- Email or notification services for sending event reminders and platform notifications.
- Internet connectivity and network services to allow communication between user devices and the server.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Eco-Volunteer platform provides a web-based user interface that allows volunteers and event organizers to interact with the system through a browser. The interface is designed to be simple, responsive, and easy to navigate so that users can quickly find and participate in environmental activities.

The main user interface components include:

- **Login and Registration Page**
  Users can create accounts or log into the system using their email and password.
- **Volunteer Dashboard**
  Displays nearby eco-events, upcoming events, and participation history.
- **Event Listing Page**
  Shows a list of available environmental activities such as clean-up drives, tree plantation events, and recycling campaigns.
- **Map Interface**
  Displays eco-events on a map so users can easily find activities near their location.
- **Event Details Page**
  Shows complete information about an event including description, location, date, organizer details, and a **"Join Event"** button.
- **Organizer Event Management Panel**
  Allows organizers to create, edit, and manage eco-events.

Users interact with the interface using:

- Buttons
- Forms
- Navigation menus
- Interactive maps
- Search bars
- Dropdown menus
- Notifications / alerts
- Sliders or filters

3.1.2 Hardware Interfaces

The Eco-Volunteer system interacts with the following hardware components:

- **User Devices**
  Smartphones, tablets, or computers used by volunteers and organizers to access the system through a web browser.
- **Network Hardware**
  Internet routers and networking equipment that enable communication between the client devices and the server.

3.1.3 Software Interfaces

The Eco-Volunteer platform interacts with several external software components:

- **Database System**
  The system uses **MongoDB** to store user information, event details, and volunteer participation records.
- **Web Browser Interface**
  Users interact with the system through modern web browsers such as Google Chrome, Firefox, or Microsoft Edge.
- **Interactive maps**
  The system uses **React Leaflet** to display an interactive map where users can view locations through markers and navigate using zoom and pan controls to easily identify places related to events or activities.
- **Notification Services**
  The system may use email or push notification services to inform volunteers about upcoming events or updates.

3.2 Functional Requirements

3.2.1 F1: User Registration

The system shall allow new users to create an account by providing basic information such as name, email address, and password.

3.2.2 F2: User Authentication

The system shall authenticate users by verifying login credentials before allowing access to the platform.

3.2.3 F3: User Profile Management

The system shall allow users to view and update their personal profile information.

3.2.4 F4: View Eco-Events

The system shall display a list of environmental events available for volunteers.

3.2.5 F5: Location-Based Event Discovery

The system shall display eco-events near the user's location using map services.

3.2.6 F6: Event Registration

The system shall allow volunteers to register for eco-events organized by NGOs or event coordinators.

3.2.7 F7: Event Creation

The system shall allow organizers to create new eco-events including event title, location, description, and date.

3.2.8 F8: Event Management

The system shall allow organizers to update or delete eco-events they have created.

3.2.9 F9: Volunteer Participation Tracking

The system shall record and maintain a history of events attended by each volunteer.

3.2.10 F10: Event Notifications

The system shall notify volunteers about upcoming events or newly created events in their area.

3.2.11 F11: Event Reports

The system shall allow organizers or administrators to view reports about event participation and volunteer engagement.

3.3 Use Case Model

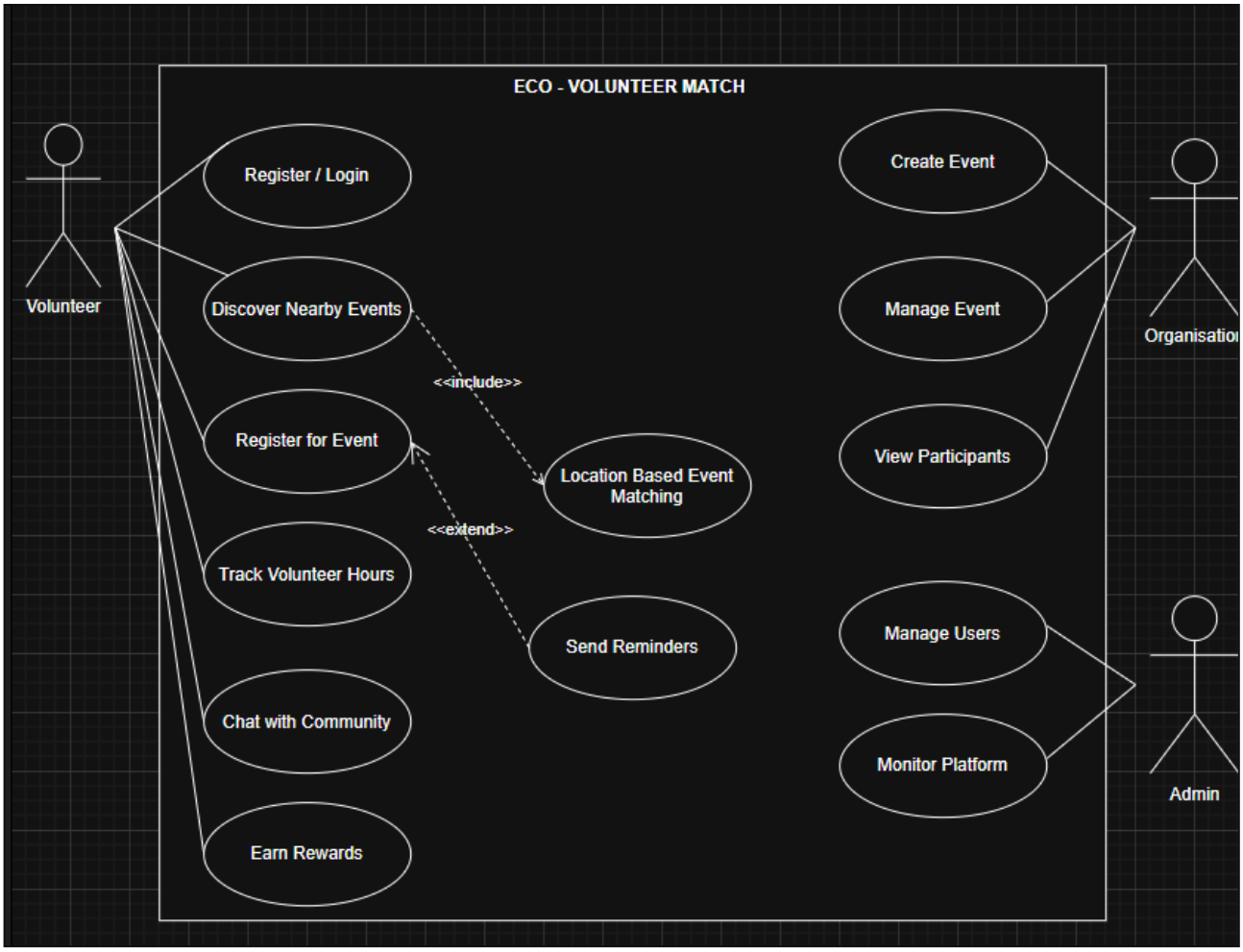

Fig 2: Use case Diagram of Eco-Volunteer Software Project

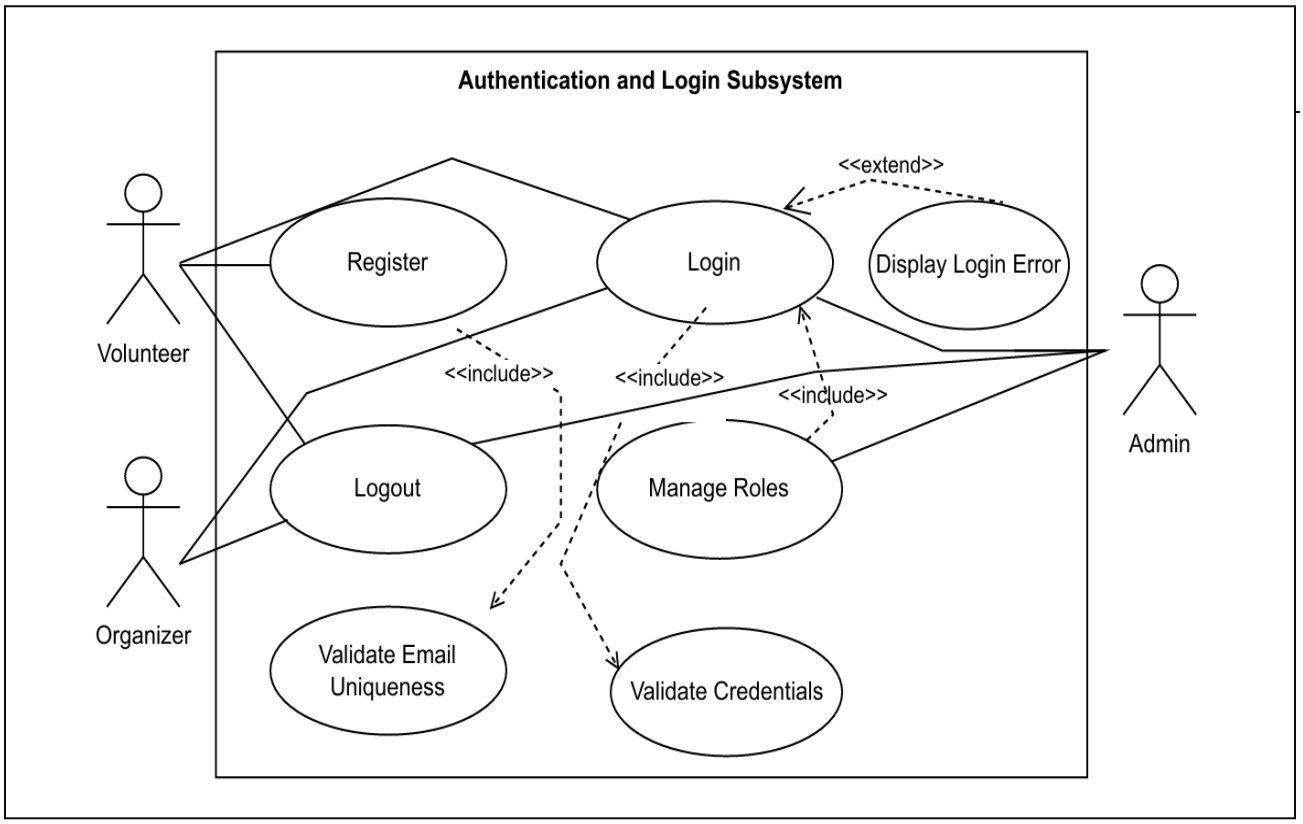

Fig 3: Use case diagram of Authentication and Login System

Author – Harika

Purpose - The purpose of this use case is to allow a registered user to access the system by verifying their login credentials. The system authenticates the user and grants access based on their assigned role.

Requirements Traceability – FR1 – User Authentication, FR2 – Credential Validation, FR3 – Role-Based Access Control

Priority – High. Authentication is a critical functionality required for secure access to the platform.

Preconditions - The user must already be registered in the system. The user must have valid login credentials (username/email and password). The system database must be accessible.

Post conditions - The user is successfully logged into the system. The system assigns the appropriate role (Volunteer, Organizer, or Admin). The user is redirected to their respective dashboard.

Actors – Volunteer, Organizer and Admin

Extends – Display Login Error (U4) – This use case extends the Login use case when authentication fails.

Flow of Events

1. Basic Flow -

The user opens the system login page.
The user enters their email/username and password.
The system validates the entered credentials.
The system verifies the user information in the database.
The system determines the user role.
The system grants access and redirects the user to the appropriate dashboard.

2. Alternative Flow -

   If the user forgets their password, they may select the Forgot Password option.
   The system allows the user to reset their password.

3. Exceptions -

   Invalid username or password.
   System database connection failure.
   User account may be inactive or blocked.

Includes

Validate Credentials (U5) – Verifies the user login information. Manage Roles (U6) – Determines and assigns the correct user role after successful login.

Notes/Issues - The system may implement additional security features such as multi-factor authentication. Password encryption should be used to ensure secure credential storage.

3.3.2 Use Case #2

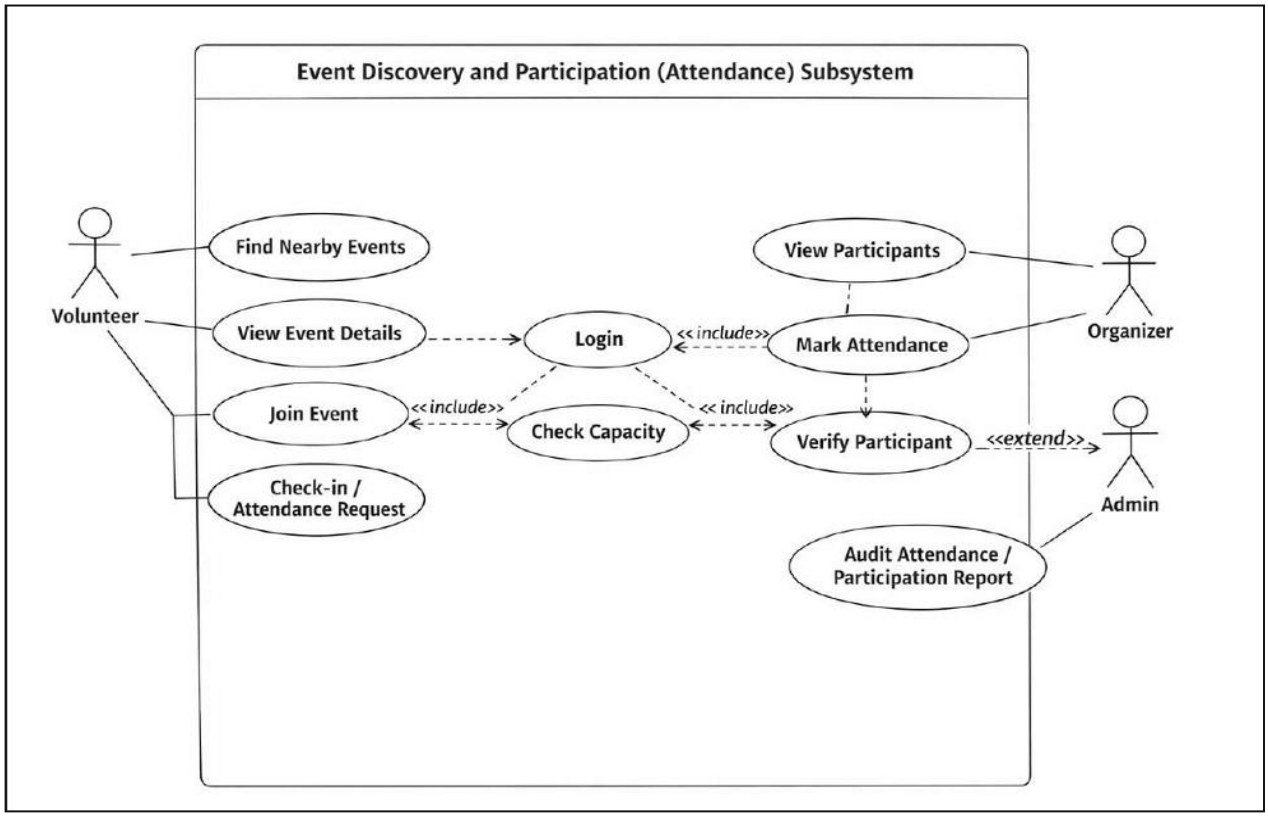

Fig 4: U se case diagram Event discovery and Participation Subsystem

Author – Manasvini
Purpose - The purpose of this use case is to allow a volunteer to register or join an available event after viewing the event details. The system verifies that the user is logged in and checks whether the event still has available capacity before confirming participation.
Requirements Traceability – FR4 – Event Discovery, FR5 – Event Participation, FR6 – Capacity Validation
Priority – High, Joining an event is a core feature of the system as it connects volunteers with environmental activities.
Preconditions - The volunteer must have an active account. The volunteer must be logged into the system. The event must exist in the system. The event must have available capacity.
Post conditions - The volunteer is successfully registered for the event. The system updates the list of participants for the event. The event capacity is reduced accordingly.
Actors – Volunteer, Organizer and Admin
Extends – None
Flow of Events

4. Basic Flow -
   The volunteer searches for nearby events.
   The system displays available events.
   The volunteer selects an event and views the event details.
   The volunteer chooses the Join Event option.
   The system checks whether the user is logged in.
   The system checks the event capacity.
   If capacity is available, the system registers the volunteer for the event.
   The system confirms successful event participation.
5. Alternative Flow -
   If the volunteer is not logged in, the system requests the user to log in first.
   After successful login, the user can continue joining the event.
6. Exceptions -
   The event is already full.
   The event has been cancelled or removed.
   System error while processing registration.

Includes

Login (U1) – Ensures the user is authenticated before joining an event. Check Capacity (U3) – Verifies whether the event still has available slots.

Notes/Issues - The system should prevent duplicate registrations for the same event. Notifications may be sent to confirm the volunteer's participation.

4 Other Non-functional Requirements

4.1 Performance Requirements

Performance requirements define how efficiently the Eco-Volunteer system should operate under different conditions.

P1. Response Time
The system shall display the list of eco-events within **3 seconds** after a user submits a search request.

P2. Map Loading Performance
The system shall load and display the map with nearby events within **4 seconds** after the user opens the map interface.

P3. Concurrent Users
The system shall support at least **500 simultaneous users** accessing the platform without significant performance degradation.

P4. Event Registration Processing
The system shall process volunteer event registrations within **2 seconds** after the user submits the registration request.

P5. Data Retrieval Performance
The system shall retrieve user profiles and event details from the database within **2 seconds** under normal operating conditions.

P6. System Availability
The system shall maintain an uptime of at least **99% during operational hours** to ensure continuous access for volunteers and organizers.

4.2 Safety and Security Requirements

- **S1. User Authentication**
  The system shall require users to authenticate using a valid **email and password** before accessing their account.

- **S2. Secure Data Transmission**
  All communication between the user device and the server shall use **HTTPS encryption** to protect sensitive information.

- **S3. Data Privacy Protection**
  Personal information such as email addresses and contact details shall be securely stored in the database and only accessible to authorized users.

- **S4. Role-Based Access Control**
  The system shall provide different access levels for **volunteers, organizers, and administrators** to prevent unauthorized actions.

- **S5. Protection Against Unauthorized Access**
  The system shall implement security measures such as **input validation, authentication checks, and session management** to prevent unauthorized access.

- **S6. Backup and Recovery**
  The system shall maintain **regular database backups** to prevent data loss in case of system failure.

- **S7. Secure Mobile/Web Access**
  Connections from mobile devices or web browsers shall be authenticated and encrypted to ensure secure interaction with the platform.

4.3 Software Quality Attributes

4.3.1 Reliability

The system shall operate reliably with minimal failures during normal usage.

- The system shall maintain **99% system uptime**.
- The system shall handle unexpected errors gracefully without crashing.
- Data stored in the system shall be protected through **automatic backups and recovery mechanisms**.

4.3.2 Usability

The platform shall be easy to learn and use for volunteers and organizers.

- The system shall provide a **simple and intuitive user interface**.
- Navigation menus and buttons shall be clearly labeled.
- Users shall be able to complete common tasks (such as joining an event) in **no more than three steps**.
- The interface shall be responsive and usable on **both desktop and mobile devices.**

4.3.3 Maintainability

The system shall be designed in a way that allows easy modification and maintenance.

- The system shall use **modular architecture** separating frontend, backend, and database components.
- Code shall follow standard **programming and documentation practices** to simplify maintenance.
- New features such as additional event types or integrations can be added without affecting existing functionality.

4.3.4 Scalability

The system shall support future growth in users and events.

- The architecture shall allow expansion to support **thousands of users and events**.
- Cloud-based hosting can be used to increase system resources when required.

5 Other Requirements

This section describes additional requirements for the Eco-Volunteer platform that are not covered in previous sections of the Software Requirements Specification. These requirements include database requirements, legal considerations, and system expansion possibilities.

5.1 Database Requirements

The Eco-Volunteer system will use a database to store and manage all platform data. The database will be implemented using **MongoDB** to support flexible and scalable data storage.

The database will store the following information:

- **User Information** – volunteer and organizer profiles including name, email, contact details, and login credentials.

- **Event Information** – event title, description, location, date, organizer details, and participation limits.
- **Participation Records** – records of volunteers who have registered or participated in events.
- **System Logs** – activity logs used for monitoring system performance and troubleshooting.

The database must support **secure storage, fast retrieval of data, and regular backup mechanisms** to prevent data loss.

5.2 Legal and Privacy Requirements

The system must comply with general data protection and privacy principles when handling user information.

- Personal information collected from users shall be used only for platform functionality.
- User data shall not be shared with third parties without consent.
- The platform shall provide a privacy policy describing how user data is collected, stored, and used.

5.3 Reusability and Future Expansion

The system will be designed to support future improvements and reuse of software components.

- Modular architecture will allow developers to add new features such as mobile applications or additional environmental programs.
- The platform may integrate additional services such as **social media sharing, environmental impact tracking, or reward systems for volunteers**.
- Interoperability interfaces may be developed to allow other environmental organizations to integrate with the Eco-Volunteer platform.

Appendix A – Data Dictionary

Variable	Description
User	Volunteers who are interested in participating in events
Event	Event/Program/Drive conducted by Eco-Organisations
Event Location	Location of eco event
Participation Status	Status of volunteer participation updated in website
Event Date	Date of event under the details provided by Eco-Organisations
U	Use Case Identifiers
FR	Functional Requirements

Appendix B - Group Log

Group Activities

- Conducted discussions to identify system requirements and features.
- Designed the high-level architecture of the Eco-Volunteer platform.
- Divided responsibilities among group members for writing different sections of the SRS.
- Reviewed and edited the document to ensure clarity, consistency, and adherence to IEEE SRS guidelines.